Nonprofit Event & Volunteer Management Platform

Software Engineering Consulting Proposal

Prepared for nonprofit and community-focused organizations

1. Executive Summary

We propose an open-source, responsive web application that centralizes nonprofit event creation, volunteer registration, shift scheduling, task assignment, attendance, and communications. The solution is intentionally lightweight so that small nonprofits can operate it without a dedicated IT department, while its architecture remains extensible for future organizations and student developers. The application will provide a responsive and accessible front end for desktop, tablet, and mobile devices, with simple navigation, clear forms, readable layouts, keyboard accessibility, and screen-reader compatibility. Coordinators and administrators will be able to manage events, shifts, tasks, volunteer assignments, and attendance, while volunteers can register for events, view available shifts, manage their availability, review schedules, and receive important event information from their preferred device.

The platform will provide isolated organizational workspaces, server-side role-based authorization, a relational database, responsive and accessible interfaces, automated email notifications, event and shift capacity controls, attendance and service-hour reporting, minimal data collection, and open-source licensing. These features will provide organizations with the tools needed to manage volunteers and events while maintaining appropriate separation of organizational data and controlling access based on user roles and permissions. The application will also be designed to accommodate users with varying levels of technical experience, from coordinators primarily working on desktop computers to volunteers accessing the platform through mobile devices.

The recommended implementation is a purpose-built Python/Flask application backed by PostgreSQL. Rather than directly extending an existing event or CRM product, the platform should borrow proven interaction patterns where useful while keeping the core domain model purpose-built for volunteer shifts. This avoids inheriting unrelated complexity and maintenance obligations while providing a focused foundation that can be expanded as organizational needs evolve.

2. About the Client

The client organizations are nonprofit and community-focused organizations that rely on volunteers to support programs, events, and services. Organizations such as Seton Center and Frederick Rescue Mission serve their communities through activities that require coordination between staff, volunteers, and community participants. Their operations can involve multiple events, volunteer roles, schedules, and participation requirements.

These organizations may have users with different levels of technical experience and different preferred methods of accessing information. Some staff and volunteers may primarily use desktop computers and email, while others may rely more heavily on mobile devices. The organizations therefore benefit from clear, straightforward tools that can be used by a broad range of users without requiring extensive technical knowledge.

As nonprofit organizations, maintaining responsible stewardship of resources and protecting volunteer information are important considerations. Their operations can also involve changing volunteer availability, event schedules, assignments, and participation records. Supporting these activities requires an approach that accommodates the practical needs of nonprofit staff and volunteers while remaining appropriate for organizations with limited technical and administrative resources.

3. Solution Overview

The proposed solution is a centralized, open-source web application for nonprofit event and volunteer management. It will bring event creation, volunteer registration, shift scheduling, task assignment, attendance, and communication into one system, reducing the need to coordinate these activities through separate spreadsheets, email threads, and other disconnected tools. Each organization will have its own workspace and controlled access to its information, while the system can support collaboration between organizations when an event requires it.

The application will provide separate experiences for administrators, coordinators, and volunteers. Administrators and coordinators will be able to create events, define volunteer positions and capacities, manage registrations, assign volunteers, track attendance, and update schedules. Volunteers will be able to create accounts, browse and filter available events and shifts, register for opportunities, manage availability, review schedules, and view attendance or service-hour information. The system can support both direct registration and registration that requires coordinator approval.

The front end will be responsive and accessible across desktop, tablet, and mobile devices. The interface will emphasize simple navigation, readable content, clear forms, and straightforward dashboards. Event and shift information can be presented through calendar and mobile-friendly list views, with filtering based on dates, availability, and event information. Authorized users will receive updates as event information, schedules, assignments, or attendance records change.

Communication and attendance will be integrated into the event workflow. The system will support automated email messages for registration confirmations, schedule updates, reminders, password recovery, and event changes. Attendance and check-in information will allow organizations to maintain participation records and service-hour histories. Volunteer information will be collected and exposed according to organizational permissions, with attention to minimal data collection, privacy, and support for requirements such as under-18 chaperone confirmation.

The initial scope will focus on the core needs of event and volunteer management rather than attempting to reproduce the full feature set of a large nonprofit CRM or event platform. The application

will establish a lightweight foundation for scheduling, communication, attendance, permissions, accessibility, and secure data handling. Its open-source structure will also allow future student developers and participating organizations to maintain and extend the system. A relational data model will support the relationships among organizations, users, events, shifts, registrations, assignments, and attendance records.

4. Technology Considerations

The technology approach is intended to keep the application lightweight, maintainable, secure, and accessible. The proposed stack emphasizes open-source technologies, relational data integrity, responsive web access, and straightforward maintenance while supporting the core requirements identified for the project.

Front End

The front end will use HTML5, CSS3, and JavaScript to provide a clean, responsive interface. The application will support desktop, tablet, and mobile use, with simple navigation, readable content, clear forms, and layouts that accommodate users with different levels of technical experience. Accessibility considerations will include keyboard navigation, screen-reader support, and interfaces that remain practical on smaller or older devices.

Back End

The recommended back end is Python with the Flask framework. Flask provides a lightweight application structure for handling authentication, user sessions, permissions, event and shift management, volunteer registration, attendance, communications, and database interactions. Authorization will be enforced on the server so that users can only access functions and information permitted by their organizational role.

Database

PostgreSQL is recommended as the primary relational database. The database will connect organizations, users, events, shifts, assignments, registrations, and attendance records while supporting constraints and transactions that protect data integrity. A relational structure is particularly appropriate for preventing duplicate registrations, controlling event and shift capacity, and maintaining consistent relationships between volunteers and scheduled activities.

Security and Data Protection

Security will be treated as a core technology consideration because the platform will contain volunteer contact and participation information. The application should use secure password hashing, protected sessions or cookies, HTTPS/TLS, parameterized database queries, and server-side role-based authorization. Data collection should remain limited to information needed for event and volunteer coordination, with organizational permissions controlling access to volunteer information.

Communication and Integrations

Email will support registration confirmations, password recovery, shift reminders, schedule updates, event cancellations, and other system-critical notifications. SMS may also be useful as a

communication option, particularly for reminders and event-day coordination. Calendar synchronization and other external integrations may be considered as later extensions rather than requirements for the initial release.

Open-Source and Maintainability

The application will use an open-source technology foundation and be structured so that future student developers and organizations can continue development. Clear documentation, organized application components, database migration practices, and a consistent architecture will be important because the platform is expected to evolve beyond its initial development period.

Deployment Considerations

Hosting and hosting-related services are outside the scope of this proposal. The application should therefore avoid unnecessary dependence on a single hosting provider and should be structured for deployment in an environment selected and managed by the client. Deployment documentation and basic operational guidance should be included so the application can be installed and maintained in the selected environment.

5. Existing Solutions

OpenVolunteer, OpenEMM, CiviCRM, Coalesce, OSEM, and Hi.Events were considered as existing solutions, along with other nonprofit and community-oriented systems. These platforms provide useful examples of event management, scheduling, volunteer coordination, communication, or nonprofit administration, but gaps remain between their existing functionality and the specific combination of volunteer shifts, organizational separation, accessibility, communications, and lightweight operation required for this project.

OpenVolunteer was identified as having strong similarities to the proposed system, including multi-organization support, event creation, volunteer scheduling, and an organized interface. However, deployment complexity and a lack of integrated communication are limitations. Coalesce is also a useful reference because it supports scheduling and live volunteer check-ins, but its limited recent maintenance is a concern.

OpenEMM is primarily an email and marketing automation platform and does not provide the required volunteer registration, shift scheduling, interactive calendar, or role-based access functionality. CiviCRM provides broader nonprofit contact and event management capabilities, but its administrative overhead, host-CMS dependency, and larger feature set are potential mismatches for the intended users. OSEM and Hi.Events provide useful event-management functionality, but their focus on conference or ticketing-related capabilities is broader or different from the project's volunteer shift requirements.

Based on this evaluation, the proposed direction is a purpose-built, lightweight application rather than directly adapting one of these existing platforms. Existing projects can still provide useful interface patterns, architectural ideas, and reference implementations, while the proposed system remains focused on the specific event, volunteer, scheduling, communication, attendance, and organizational-access requirements.

6. Preliminary Schedule

The schedule below outlines the proposed phases of the project. Exact dates should be adjusted after the client confirms requirements, technology decisions, and project resources.

Phase	Approx. Timing	Primary Activities	Major Deliverable
Planning	2 weeks	Finalize requirements, scope, project resources, technology decisions, and priorities.	Approved requirements and project plan
Design	2 weeks	Develop interface structure, database design, architecture, permissions, and security approach.	UI designs and technical design
Implementation	3–4 weeks	Build core event, volunteer, scheduling, communication, attendance, and role-management functionality.	Working application with core features
Testing and Integration	2 weeks	Test functionality, security, accessibility, usability, data handling, and integration between application components.	Tested release candidate
Final Review and Deployment Preparation	1 week	Resolve final issues, complete documentation, verify deployment behavior, and prepare for client review.	Client-ready application and documentation

This proposal places requirements and planning first, followed by design, implementation, testing, and final deployment or client review. The implementation phase should prioritize core event and volunteer functions before secondary features so that testing time is protected.

7. Risks & Assumptions

Risks

Volunteer data security: Improper handling of volunteer contact or participation information could create security and privacy concerns.

Mitigation: Use secure password handling, server-side authorization, protected sessions, HTTPS/TLS, parameterized queries, minimal data collection, and clear organizational access boundaries.

Scope growth: Adding too many secondary features at the same time could overwhelm the development schedule and reduce testing time.

Mitigation: Prioritize event creation, volunteer registration, scheduling, communication, attendance, permissions, accessibility, and secure data handling before secondary enhancements.

Organizational data separation: Multiple organizations require strong separation while some events may require controlled collaboration between organizations.

Mitigation: Define organization-level boundaries and permissions early and explicitly control any shared event or partner access.

Different technical skill levels: Organizers, volunteers, and administrators may have very different levels of technical experience.

Mitigation: Keep volunteer and organizer workflows simple, provide more detailed controls to administrators, and design for desktop, mobile, accessibility, and older devices.

Technology changes: Changing the selected programming language, framework, database, or architecture after implementation begins could delay the project.

Mitigation: Finalize major technology decisions during planning and maintain a clear technical design.

Third-party services: Email, SMS, or other external services may introduce usage limits, configuration requirements, or dependency risks.

Mitigation: Keep integrations limited to required functions and manage communication features separately from core event data.

Late or changing requirements: Changes to partner information, event requirements, or client preferences can affect implementation and testing.

Mitigation: Confirm requirements during planning and use client review points during design, implementation, and final testing.

Open-source maintenance: Future contributors may have difficulty understanding the project if architecture and development practices are unclear.

Mitigation: Provide documentation and maintain a consistent, organized application structure.

Assumptions

- The application will be responsive across desktop, tablet, and mobile devices, with desktop use supporting administrative and back-office work and mobile access supporting volunteers.
- An open-source foundation is required so future student developers and participating organizations can maintain and extend the application.
- Organizations will provide accurate information needed to configure events, volunteer roles, schedules, and registration requirements.

- Organizations will have personnel and devices capable of managing the application and its event information.
- Organizations are responsible for obtaining the appropriate permissions to collect and store volunteer information.
- The application will collect only information necessary for the proposed event and volunteer management functions; highly sensitive information such as Social Security numbers or payment information is not part of the stated initial requirements.
- The client will select and manage the hosting environment; hosting services themselves are outside the scope of this proposal.
- Exact visual branding, theme, and detailed design preferences remain subject to client confirmation.
- Specific permission levels and rules for controlled sharing between organizations will be finalized during requirements and design.
- Strict age verification is not an initial requirement; where applicable, a simple age-related acknowledgment and chaperone requirement can be used.